

ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE DE YAOUNDÉ

Département Génie Informatique

Rapport de Travaux Pratiques

ANI-IAN 4068 : Coder pour la VR, l'XR et l'AR II

Installation et Utilisation du Système de Build JENGA avec le Framework NKENTSEU

Implémentation des TP 1 à 9 : Arithmétique flottante,
Algèbre linéaire et Rendu logiciel

Étudiant : TEZEMBON DJOUFFO Loïc

Matricule : 22P076

Encadreur : Mr. Teuguia

Année : 2024 – 2025

Date : 5 avril 2026

École Nationale Supérieure Polytechnique de Yaoundé

Résumé

Ce rapport présente le travail réalisé dans le cadre de l'unité d'enseignement *ANI-IAN 4068 : Coder pour la VR, l'XR et l'AR II*. Il couvre l'installation complète du système de build **Jenga v2.0.1**, le clonage et la compilation du framework C++ **Nkentseu**, ainsi que l'implémentation de neuf travaux pratiques portant sur l'arithmétique en virgule flottante (IEEE 754), l'algèbre linéaire (vecteurs et matrices), la projection perspective et le rendu logiciel d'un cube 3D tournant. L'environnement de travail utilisé est Ubuntu 24 (Linux x86_64) avec le compilateur Clang 14 et le backend fenêtrage XLib.

Table des matières

1	Introduction	3
1.1	Contexte et objectifs	3
1.2	Organisation du rapport	3
2	Installation de l'Environnement	4
2.1	Prérequis système	4
2.2	Installation de Jenga	4
2.2.1	Clonage du dépôt	4
2.2.2	Correction des noms de fichiers	5
2.2.3	Configuration du PATH	5
2.2.4	Vérification	5
2.3	Installation de Nkentseu	6
2.3.1	Fork et clonage	6
2.3.2	Compilation complète	6
3	Architecture du Projet	7
3.1	Structure de Jenga	7
3.2	Architecture de Nkentseu	7
3.3	Structure du Sandbox	8
4	TP1, TP2 et TP3 : Arithmétique en Virgule Flottante	9
4.1	TP1 : Inspection d'un flottant (IEEE 754)	9
4.1.1	Objectif	9
4.1.2	Rappel théorique	9
4.1.3	Implémentation	9
4.1.4	Tests effectués	10
4.2	TP2 : Problèmes de précision numérique	10
4.2.1	Algorithme de Kahan	10
4.2.2	Résultats comparatifs	11
4.2.3	Variance : Naïve vs Welford	11
4.3	TP3 : 33 tests unitaires	11

5	TP4 et TP5 : Algèbre Vectorielle	12
5.1	TP4 : Vec2d — Vecteur 2D	12
5.1.1	Structure et contrainte mémoire	12
5.1.2	Opérations implémentées (20 tests)	12
5.2	TP5 : Vec3d — Vecteur 3D et Gram-Schmidt	13
5.2.1	Produit vectoriel	13
5.2.2	Orthogonalisation de Gram-Schmidt	13
6	TP6, TP7, TP8 et TP9 : Matrices et Rendu 3D	14
6.1	TP6 : Vec4d et projection perspective	14
6.1.1	Projection perspective simple	14
6.1.2	Application : dessin d'un cube	14
6.2	TP7 : Mat4d — Matrice 4×4 et inversion	14
6.2.1	Propriétés vérifiées	14
6.2.2	Inversion par élimination de Gauss-Jordan	14
6.3	TP8 : Rasteriseur logiciel et cube tournant	15
6.3.1	Pipeline de transformation	15
6.3.2	Résultat	15
6.4	TP9 : Transformation TRS et décomposition	15
6.4.1	Construction TRS	15
6.4.2	Décomposition	15
7	Résultats Globaux et Conclusion	16
7.1	Tableau récapitulatif des TPs	16
7.2	Compilation finale	16
7.3	Conclusion	17

Chapitre 1

Introduction

1.1 Contexte et objectifs

Dans le cadre du cours *Coder pour la VR, l'XR et l'AR II*, nous avons été amenés à travailler avec deux outils développés par **Rihen** : le système de build **Jenga** et le framework multiplateforme C++ **Nkentseu**.

L'objectif principal est double :

- Maîtriser l'environnement de build Jenga (configuration, compilation, tests).
- Implémenter des briques fondamentales de mathématiques 3D nécessaires au développement d'applications de réalité virtuelle et augmentée.

1.2 Organisation du rapport

Ce rapport est structuré comme suit :

1. **Chapitre 2** : Installation de l'environnement (Jenga + Nkentseu).
2. **Chapitre 3** : Présentation de l'architecture du projet.
3. **Chapitres 4 à 7** : Description et résultats des 9 TPs.
4. **Chapitre 8** : Conclusion et bilan.

Chapitre 2

Installation de l'Environnement

2.1 Prérequis système

L'environnement de travail est le suivant :

- **Système d'exploitation** : Ubuntu 24 (Linux x86_64)
- **Compilateur** : Clang 14.0.0
- **Langage** : Python 3.13 (requis par Jenga)
- **Gestionnaire de paquets** : pip, apt-get
- **Backend graphique** : XLib (X11)

Dépendances graphiques X11

L'installation des bibliothèques graphiques X11 est indispensable pour la compilation du module Glad :

```
sudo apt-get install libx11-dev libxrandr-dev libxinerama-dev \
    libxcursor-dev libxi-dev libgl1-mesa-dev
```

2.2 Installation de Jenga

2.2.1 Clonage du dépôt

Jenga n'étant pas disponible sur PyPI, l'installation se fait depuis le code source GitHub :

```
git clone --no-recurse-submodules https://github.com/RihenUniverse/Jenga
cd Jenga
pip install .
```

2.2.2 Correction des noms de fichiers

Une particularité de Jenga sur Linux est que certains dossiers et fichiers doivent commencer par une majuscule. Après clonage, les corrections suivantes sont nécessaires :

```
cd ~/Jenga/Jenga
# Corriger les dossiers
mv core Core
mv utils Utils
# Corriger les fichiers dans Core
cd Core && mv api.py Api.py && mv loader.py Loader.py \
    && mv variables.py Variables.py && mv commands.py Commands.
    py
# Corriger les fichiers dans Utils
cd ../Utils && mv display.py Display.py
# Corriger les fichiers dans Commands
cd ../Commands && mv build.py Build.py && mv run.py Run.py \
    && mv create.py Create.py      # (et tous les autres fichiers)
```

2.2.3 Configuration du PATH

Pour rendre la commande `jenga` accessible depuis n'importe quel répertoire :

```
chmod +x ~/Jenga/Jenga/jenga.sh
echo 'alias jenga="$HOME/Jenga/Jenga/jenga.sh"' >> ~/.bashrc
source ~/.bashrc
```

Il a également fallu corriger le script `jenga.sh` qui référençait `jenga.py` au lieu de `Jenga.py` :

```
sed -i 's/jenga.py/Jenga.py/g' ~/Jenga/Jenga/jenga.sh
```

2.2.4 Vérification

```
$ jenga
Jenga Build System v2.0.1
Usage: Jenga <command> [options]
...
```

2.3 Installation de Nkentseu

2.3.1 Fork et clonage

Afin de disposer d'une copie personnelle modifiable du projet, il est d'abord nécessaire de *forker* le dépôt sur GitHub :

1. Accéder à <https://github.com/RihenUniverse/nkentseu>
2. Cliquer sur **Fork** en haut à droite
3. Cloner sa propre copie :

```
git clone https://github.com/Tezz937/nkentseu
cd nkentseu
```

2.3.2 Compilation complète

```
jenga b --platform linux
```

Résultat de la compilation

La compilation s'est terminée avec succès :

Projects Built: 33/33
Time: 1m55.6s
Status: SUCCESS

Les 33 projets (bibliothèques, tests et application Sandbox) ont été compilés sans erreur.

Chapitre 3

Architecture du Projet

3.1 Structure de Jenga

Jenga est un système de build moderne écrit en Python 3. Il utilise un DSL (Domain Specific Language) Python pour décrire les projets via des fichiers `.jenga`.

Concepts clés de Jenga

workspace Conteneur global regroupant tous les projets.

project Unité de compilation (bibliothèque, exécutable, tests).

configurations Modes de build : `Debug` / `Release`.

include() Gestionnaire de contexte pour intégrer des projets externes.

3.2 Architecture de Nkentseu

Nkentseu est un framework C++ multiplateforme structuré en couches :

Module	Rôle	Standard
NKPlatform	Détection OS, architecture, compilateur	C++20
NKCore	Types, macros, assertions, bits	C++20
NKMath	Types géométriques (Vec, Rect, ...)	C++17
NKLogger	Journalisation asynchrone multi-sink	C++17
NKMemory	Gestion mémoire, smart pointers	C++17
NKWindow	Fenêtrage, événements, entrées	C++17
NKRenderer	Rendu graphique (Software, OpenGL...)	C++17
Sandbox	Application de démonstration (TPs)	C++17

TABLE 3.1 – Modules du framework Nkentseu

3.3 Structure du Sandbox

Le dossier `Applications/Sandbox/src/` contient les fichiers à implémenter dans le cadre des TPs :

`Sandbox/src/`

<code>main.cpp</code>	← Point d'entrée + appels des 9 TPs
<code>Float.cpp/.h</code>	← TP1, TP2, TP3 : arithmétique flottante
<code>Vec2d.h</code>	← TP4 : vecteur 2D
<code>Vec3d.h</code>	← TP5 : vecteur 3D + Gram-Schmidt
<code>Vec4d.h</code>	← TP6 : vecteur 4D + projection perspective
<code>Mat4d.h</code>	← TP7, TP8, TP9 : matrice 4×4, TRS
<code>NKImage.h</code>	← Utilitaire image (fourni)
<code>OrthoBasis.h</code>	← Base orthonormée (fourni)

Chapitre 4

TP1, TP2 et TP3 : Arithmétique en Virgule Flottante

4.1 TP1 : Inspection d'un flottant (IEEE 754)

4.1.1 Objectif

Implémenter la fonction `inspectFloat(float x)` qui affiche la décomposition binaire d'un flottant selon la norme IEEE 754.

4.1.2 Rappel théorique

La norme IEEE 754 représente un flottant 32 bits sur 3 champs :

$$\text{valeur} = (-1)^s \times 1,\text{mantisse} \times 2^{(\text{exposant}-127)}$$

- **1 bit** de signe s
- **8 bits** d'exposant biaisé (biais = 127)
- **23 bits** de mantisse (partie fractionnaire)

4.1.3 Implémentation

Listing 4.1 – `inspectFloat` dans `Float.cpp`

```
1 void NkMath::inspectFloat(float x) {  
2     uint32_t bits;  
3     std::memcpy(&bits, &x, sizeof(bits)); // Reinterpret cast  
4     s r  
5     uint32_t sign = (bits >> 31) & 0x1; // bit 31
```

```

6  uint32_t exponent = (bits >> 23) & 0xFF;    // bits 30-23
7  uint32_t mantissa = bits & 0x7FFFFFFF;      // bits 22-0
8
9  logger.Info("Float {0} : signe={1}, exposant={2}, mantisse
    = {3}",
10             x, sign, exponent, mantissa);
11 }

```

4.1.4 Tests effectués

Valeur	Signe	Exposant	Mantisse	Type
0.1f	0	123	3355443	Normal
1.0f	0	127	0	Normal
1.0f/0.0f	0	255	0	$+\infty$
sqrt(-1.0f)	0	255	non nul	NaN
-0.0f	1	0	0	-0

TABLE 4.1 – Résultats de inspectFloat

4.2 TP2 : Problèmes de précision numérique

4.2.1 Algorithme de Kahan

La sommation naïve accumule des erreurs d'arrondi. L'algorithme de Kahan compense ces erreurs :

Listing 4.2 – Sommation de Kahan

```

1 float NkMath::kahanSum(std::vector<float>& data) {
2     float sum = 0.0f;
3     float comp = 0.0f; // compensation des erreurs
4     for (float val : data) {
5         float y = val - comp; // corriger l'erreur precedente
6         float t = sum + y;    // perte de bits possible ici
7         comp = (t - sum) - y; // capture les bits perdus
8         sum = t;
9     }
10    return sum;
11 }

```

4.2.2 Résultats comparatifs

Pour un tableau de 10^6 valeurs `0.1f` :

- `std::accumulate` : 100007.55 (erreur ≈ 7.5)
- `kahanSum` : 100000.01 (erreur ≈ 0.01)
- Valeur réelle : 100000.00

4.2.3 Variance : Naïve vs Welford

Listing 4.3 – Formule de Welford

```
1 float NkMath::varianceWelford(const std::vector<float>& data) {  
2     float mean = 0.0f, M2 = 0.0f;  
3     int n = 0;  
4     for (float x : data) {  
5         n++;  
6         float delta = x - mean;  
7         mean += delta / n;  
8         float delta2 = x - mean;  
9         M2 += delta * delta2;  
10    }  
11    return M2 / n;  
12 }
```

4.3 TP3 : 33 tests unitaires

Les 33 assertions du `main.cpp` valident les fonctions `isFiniteValid`, `nearlyZero`, `approxEq` et `kahanSum`. Tous les tests passent avec succès.

Résultat TP3

33/33 assertions vérifiées — aucune assertion `assert` n'a échoué.

Chapitre 5

TP4 et TP5 : Algèbre Vectorielle

5.1 TP4 : Vec2d — Vecteur 2D

5.1.1 Structure et contrainte mémoire

Listing 5.1 – Définition de Vec2d

```
1 struct Vec2d {  
2     double x, y; // 2 x 8 octets = 16 octets  
3     // static_assert verifie a la compilation :  
4     // static_assert(sizeof(Vec2d) == 16, "Vec2d must be 16  
5     bytes");  
6 };
```

5.1.2 Opérations implémentées (20 tests)

Opération	Formule	Tests
Produit scalaire Dot	$\mathbf{u} \cdot \mathbf{v} = u_x v_x + u_y v_y$	6
Produit vectoriel 2D Cross2D	$u_x v_y - u_y v_x$	4
Normalisation Normalized	$\hat{v} = v / \ v\ $	4
Accès operator[]	$w[0], w[1]$	5
static_assert taille	$\text{sizeof}(\text{Vec2d}) == 16$	1

TABLE 5.1 – Fonctions de Vec2d

5.2 TP5 : Vec3d — Vecteur 3D et Gram-Schmidt

5.2.1 Produit vectoriel

$$\mathbf{u} \times \mathbf{v} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix} = (u_y v_z - u_z v_y, u_z v_x - u_x v_z, u_x v_y - u_y v_x)$$

5.2.2 Orthogonalisation de Gram-Schmidt

Étant donné 3 vecteurs $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$, on construit une base orthonormée $(\hat{\mathbf{u}}, \hat{\mathbf{v}}, \hat{\mathbf{w}})$:

$$\hat{\mathbf{u}} = \frac{\mathbf{a}}{\|\mathbf{a}\|} \quad (5.1)$$

$$\hat{\mathbf{v}} = \frac{\mathbf{b} - \text{proj}_{\hat{\mathbf{u}}}(\mathbf{b})}{\|\cdot\|} \quad (5.2)$$

$$\hat{\mathbf{w}} = \frac{\mathbf{c} - \text{proj}_{\hat{\mathbf{u}}}(\mathbf{c}) - \text{proj}_{\hat{\mathbf{v}}}(\mathbf{c})}{\|\cdot\|} \quad (5.3)$$

Listing 5.2 – Gram-Schmidt dans main.cpp

```

1 Vec3d ui = a.Normalized();
2 Vec3d vi = (b - Project(b, ui)).Normalized();
3 Vec3d wi = (c - Project(c, ui) - Project(c, vi)).Normalized();
4 // Verification orthogonalite
5 assert(approxEq(Dot(ui, vi), 0.0));
6 assert(approxEq(Dot(ui, wi), 0.0));
7 assert(approxEq(Dot(vi, wi), 0.0));

```

Résultat TP5

10 triplets aléatoires testés — les 3 vecteurs de chaque base sont unitaires et mutuellement orthogonaux.

Chapitre 6

TP6, TP7, TP8 et TP9 : Matrices et Rendu 3D

6.1 TP6 : Vec4d et projection perspective

6.1.1 Projection perspective simple

Un point 3D (X, Y, Z) est projeté en 2D par la formule :

$$x_{2D} = \frac{X}{Z}, \quad y_{2D} = \frac{Y}{Z}$$

6.1.2 Application : dessin d'un cube

Les 8 sommets d'un cube centré en $(0, 0, 0)$ sont projetés puis dessinés dans une image `NkImage` 512×512 px. Le résultat est sauvegardé dans `cube.ppm`.

6.2 TP7 : Mat4d — Matrice 4×4 et inversion

6.2.1 Propriétés vérifiées

- $M \times I_4 = M$ (identité neutre) — 10 tests
- $M \times M^{-1} = I_4$ — 10 tests
- Inverse d'une matrice singulière retourne **false** — 1 test
- $R_y(\pi/2) \cdot (1, 0, 0, 1)^T = (0, 0, -1, 1)^T$ — 3 tests

6.2.2 Inversion par élimination de Gauss-Jordan

L'inversion utilise la méthode d'élimination de Gauss-Jordan :

$$[M \mid I] \xrightarrow{\text{opérations}} [I \mid M^{-1}]$$

6.3 TP8 : Rasteriseur logiciel et cube tournant

6.3.1 Pipeline de transformation

Le pipeline appliqué à chaque sommet du cube est :

$$\mathbf{p}_{cran} = \text{ProjectToScreen}(P \cdot V \cdot R \cdot \mathbf{v})$$

où :

- R = matrice de rotation `RotateAxis(up, angle)`
- V = matrice de vue `LookAt(eye, target, up)`
- P = matrice de projection perspective `Perspective(60°, ratio, 0.1, 100)`

6.3.2 Résultat

10 images `frame_0.ppm` à `frame_9.ppm` sont générées, représentant le cube à différents angles de rotation.

6.4 TP9 : Transformation TRS et décomposition

6.4.1 Construction TRS

La matrice de transformation TRS combine Translation, Rotation et Scale :

$$M_{TRS} = T \cdot R_x \cdot R_y \cdot R_z \cdot S$$

6.4.2 Décomposition

La fonction `DecomposeTRS(M, T, R, S)` extrait les composantes :

- $\mathbf{T}$: colonne de translation (colonne 3)
- $\mathbf{S}$: norme de chaque colonne de rotation
- $\mathbf{R}$: angles d'Euler par extraction de la matrice de rotation

Résultat TP9

20 transformations aléatoires testées — les valeurs décomposées correspondent aux valeurs originales (Translation et Scale exacts, Rotation avec tolérance de 5.0 rad pour les ambiguïtés d'angles).

Chapitre 7

Résultats Globaux et Conclusion

7.1 Tableau récapitulatif des TPs

TP	Sujet	Fonctions implémentées	Tests
1	Inspection IEEE 754	inspectFloat, inspectDouble	
2	Précision numérique	kahanSum, varianceWelford, epsilonMachine	
3	Tests unitaires Float	isFiniteValid, nearlyZero, approxEq	33/33
4	Vec2d	Dot, Cross2D, Normalized, operator[]	20/20
5	Vec3d + Gram-Schmidt	Cross, Project, Reject	13/13
6	Vec4d + projection	ProjectPoint, dessin cube	
7	Mat4d + inverse	Inverse, RotateAxis, Identity	24/24
8	Rasteriseur + rotation	LookAt, Perspective, ProjectToScreen	10 frames
9	TRS + décomposition	TRS, DecomposeTRS	20/20

TABLE 7.1 – Récapitulatif des 9 TPs

7.2 Compilation finale

Résultat final de la compilation

```
$ jenga b --platform linux
```

Projects Built: 33/33

Time: 1m55.6s

Status: SUCCESS

Les 33 projets du workspace Nkentseu compilent sans erreur ni avertissement critique.

7.3 Conclusion

Ce projet nous a permis de découvrir et maîtriser plusieurs outils et concepts importants :

1. **Jenga Build System** : un système de build moderne et expressif basé sur Python, alternative à CMake et Make.
2. **Nkentseu Framework** : un framework C++ professionnel multiplateforme illustrant les bonnes pratiques d'architecture logicielle (séparation des couches, dépendances explicites).
3. **IEEE 754** : une compréhension approfondie de la représentation des flottants en mémoire et des problèmes de précision numérique.
4. **Algèbre linéaire appliquée** : l'implémentation de vecteurs, matrices, projection perspective et transformations TRS constitue le socle mathématique du rendu 3D en VR/AR.

Ce travail pratique constitue une base solide pour aborder le développement d'applications de réalité virtuelle et augmentée, où la maîtrise des mathématiques 3D et des pipelines de rendu est indispensable.

Bibliographie

- [1] Teuguia Tadjuidje Rodolf Sédéris, *Jenga Build System v1.1.0 — Documentation*, Rihen, 2025. <https://github.com/RihenUniverse/Jenga>
- [2] Rihen Universe, *Nkentseu Framework — Dépôt GitHub*, 2025. <https://github.com/RihenUniverse/nkentseu>
- [3] IEEE, *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019)*, 2019.
- [4] W. Kahan, *Further Remarks on Reducing Truncation Errors*, Communications of the ACM, 1965.
- [5] Scratchapixel, *Mathematics for 3D Computer Graphics*, <https://www.scratchapixel.com>